An Exact Radial Obstruction in a Base-19 Reverse-and-Add Constraint System

Anonymous

August 2026

Abstract

We study one explicitly frozen bounded-integer constraint system extracted from a six-phase reverse-and-add topology in base 19. Exact radial frontier enumeration gives an empty terminal frontier for a 409-equation candidate and for a 408-equation subset $\mathcal{C}'$. A complete one-equation-deletion audit supplies a satisfying assignment after deletion of each equation of $\mathcal{C}'$, so $\mathcal{C}'$is deletion-minimal unsatisfiable in the computational certificate model. A standalone checker reconstructs every local quantifier-free linear-integer problem, enumerates every output projection, and compares the complete transition relation with the promoted certificate. Lean 4 checks the generic frontier soundness theorem and, separately, one concrete satisfying neighbor. The two concrete unsatisfiability results are *not kernel-checked*: they rely on exact Z3 enumeration and do not carry an LRAT-like solver proof. The result is not a minimum-cardinality-core claim, gives no all-base theorem, and does not establish any general Lychrel claim.

1 Introduction and Scope

For a base-b word, the reverse-and-add operation adds the represented integer to the integer represented by the reversed word. Carries couple columns that are adjacent in the usual direction, while reversal couples digits that are far apart. A natural way to expose the resulting structure is therefore to process columns radially, from both ends toward the center.

This paper reports a finite result about one frozen system, not about the unbounded reverse-and-add process. The topology was motivated by David Seal's reported period-six self-similar construction in base 17 [3]. The OEIS sequence A060382 records conjectural least palindrome-free starts across bases and explicitly warns that, apart from its base-two entry, its listed values are unproved [10]. Neither source is used as a premise of the result below, and the present obstruction does not settle the base-19 entry of that sequence.

Our contribution is an auditable artifact for the following scoped statement. The fixed candidate $\mathcal{C}$ is unsatisfiable, and deleting one specified equation produces a deletion-minimal unsatisfiable subset $\mathcal{C}'$. "Deletion-minimal" means that removing any one remaining equation makes this particular subset satisfiable; it does not mean that no smaller unsatisfiable subset exists. The concrete claims have exact computational replays, whereas the general soundness implication has a Lean proof. We keep these assurance levels separate throughout.

2 Fixed Constraint System

Let $b = 19$. A digit variable ranges over $D = \{0, \ldots, b-1\}$, a carry variable over $K = \{0, 1\}$, and the first digit of each phase is constrained to be nonzero. There are six phases $p \in \{0, \ldots, 5\}$. Each

phase word has three variable blocks. Phase zero has block lengths $(26, 25, 31)$; every other phase has lengths $(29, 23, 31)$. For a nonnegative parameter n, two constant runs are inserted between the blocks. Their digits are $(0, b-1)$ in phases zero and two through five, and $(b-1, 0)$ in phase one; each run has length n. Write the resulting word as $W_p(n)$.

For $p < 5$, phase p relates $W_p(0)$ to $W_{p+1}(0)$. The last phase relates $W_5(0)$ to $W_0(1)$, which is the self-similar growth step. At column k, let $\ell_{p,k}$ and $r_{p,k}$ be the two source atoms selected from opposite ends, let $t_{p,k}$ be the corresponding target atom, and let $c_{p,k-1}, c_{p,k} \in K$ be the incoming and outgoing carries. Missing end atoms and boundary carries are zero. The column equation is

$$\ell_{p,k} + r_{p,k} + c_{p,k-1} = t_{p,k} + b\, c_{p,k}. \tag{1}$$

The canonical builder retains columns 0–33 and 48–82 in phase zero; columns 0–33 and 49–82 in phases one through four; and columns 0–33 and 49–83 in phase five. It then omits phase-five column 82. This gives the candidate $\mathcal{C}$. The independent equation manifest contains exactly 409 equations over 409 active digit variables and 410 carry variables. The retained equations are a subset of the full topology, so their inconsistency obstructs that topology; no converse encoding claim is needed.

The equation manifest serializes the base, topology digest, and every atom, equation, provenance label, and equation order. Domain and leading-digit conventions are explicit in both checkers and are derived deterministically from the base and variable names; they are not separate fields in the version one JSON schema. A standard-library-only Python builder and a Lean builder produce canonical equation renderings that are byte-compared during reproduction, while construction tests check variable counts and model validity. These checks guard against statement drift; they do not themselves prove unsatisfiability.

3 Radial Decomposition

Partition the retained equations into ordered radial layers $L_0, \ldots, L_m$. At a cut after L_i, a variable is *live* precisely when it occurs in an equation at or before the cut and also in an equation after the cut. Let I_i be the canonically sorted live-variable list. A frontier state assigns values only to I_i. Variables used only inside one layer are existentially hidden.

For an input state s on I_{i-1}, define

$$T_i(s) = \{\pi_{I_i}(a) : a \text{ extends } s \text{ and satisfies every equation in } L_i\}.$$

Starting with $F_{-1} = \{()\}$, exact propagation sets $F_i = \bigcup_{s \in F_{i-1}} T_i(s)$. This is variable elimination along a path decomposition. Its cost is governed by the live interface rather than the total number of variables, echoing the classical bounded-width view of constraint satisfaction [7, 6].

For $\mathcal{C}'$, the promoted certificate contains 35 serialized layers, labeled from -1 through 33. A cut contains at most 2 live digits and 12 live carries. The largest reachable frontier has 314 states, and the output after layer 33 is empty. These figures are read from the promoted certificate into this manuscript; they are not copied as independent literals.

4 Soundness and Certification

Exact enumeration is stronger than necessary for soundness. Suppose $G_{-1}, \ldots, G_m$ are sets of interface states such that the empty input is in G_{-1} and, for every layer,

$$s \in G_{i-1} \ \wedge \ u \in T_i(s) \quad \Longrightarrow \quad u \in G_i. \tag{2}$$

Theorem 1 (closed empty frontier)**.** *If every global satisfying assignment restricts to a locally satisfying assignment in every layer, the frontiers satisfy* (2)*, and $G_m = \varnothing$, then the global constraint system is unsatisfiable.*

Proof. Assume a global satisfying assignment a. Its restriction to the empty initial interface is in G_{-1}. Inductively, local satisfaction in L_i puts $\pi_{I_i}(a)$ in $T_i(\pi_{I_{i-1}}(a))$, and closure puts that restriction in G_i. At the final layer this says $\pi_{I_m}(a) \in G_m = \varnothing$, a contradiction. □

Lean 4 formalizes this argument as `Base19Radial.unsat_of_closed_empty_frontier`. It also checks a hand-written two-layer contradiction, thereby exercising the theorem with concrete data. The reported axiom dependency is only Lean's standard `propext` principle [5].

The concrete frontier checker is deliberately separate from the exploratory generator. For each serialized layer and each input state it:

1. reconstructs the layer equations from the canonical manifest;
2. asserts digit, leading-digit, carry, and input-state bounds in QF_LIA;
3. asks Z3 for a model, records the output projection, blocks that projection, and repeats until unsatisfiable; and
4. compares the sorted projections, every input transition, live-cut variables, equation coverage, and content digests with the certificate.

Z3's integer sort denotes mathematical integers, and its documented LIA machinery uses cuts and branch-and-bound [9]. The checker rejects `unknown`; it never treats solver termination as unsatisfiability. Replaying the $\mathcal{C}'$ certificate took 34.53 seconds on the recorded development machine.

Proposition 1 (computational obstruction)**.** *Under the correctness of the manifest parser, the QF_LIA encoding, and Z3's reported results, $\mathcal{C}$ and $\mathcal{C}'$ are unsatisfiable.*

The qualifier in Proposition 1 is material. The Z3 runs do not emit a proof trace checked by a small independent proof checker, and the concrete frontier closure obligations are not discharged by the Lean kernel.

5 Deletion-Minimal Core

Let $e_\star$ be the equation `phase=0,n=0,column=82`, and set $\mathcal{C}' = \mathcal{C} \setminus \{e_\star\}$. The standalone frontier replay establishes computational unsatisfiability of $\mathcal{C}'$. A complete audit then solves every system $\mathcal{C} \setminus \{e\}$ for $e \in \mathcal{C}$. Of 409 outcomes, 408 have complete satisfying assignments and 1 is unsatisfiable; the latter is the deletion of $e_\star$.

For any $e \in \mathcal{C}'$, the stored assignment for $\mathcal{C} \setminus \{e\}$ also satisfies its subset $\mathcal{C}' \setminus \{e\}$. Direct integer replay checks all bounds and retained column equalities for every such assignment. Therefore every one-equation deletion of $\mathcal{C}'$ is satisfiable in the same computational certificate model, proving deletion minimality relative to the frozen candidate.

One neighboring result has a stronger endpoint. Deleting `phase=0,n=0,column=0` leaves 408 equations, and the full assignment is embedded as transparent Lean data. Ordinary kernel reduction checks the theorem `Base19Radial.deleteP0C0Sat` in 37.60 seconds on the recorded machine; no `native_decide` oracle is used. This kernel check covers one SAT neighbor, not the complete deletion audit.

6 Reproducibility and Trust

The artifact included with this preprint follows the computer-aided-proof principles of making the computational role explicit and supplying surveyable code and supporting data [8]. Its quick entry

Object	Equations	Assurance endpoint
Candidate	409	exact QF_LIA frontier replay
Core	408	exact replay plus all-deletion witnesses
Known SAT neighbor	408	Lean kernel, ordinary `decide`
Generic soundness theorem	—	Lean kernel

Table 1: Claim boundaries. The first two rows are not Lean theorems.

point verifies frozen SHA-256 values; regenerates the equation and Lean data; independently replays both empty frontiers and all promoted SAT witnesses; runs the publication tests; performs a clean Lean build; compares the Lean-exported and Python-built statements; and prints theorem axiom dependencies. The command is

`./publication/base19-radial/reproduce.sh quick`.

The recorded clean-clone run took 245.46 seconds. The `full` mode additionally regenerates the complete deletion audit and all promoted certificates before byte comparison. Certificate generation alone took 172.90 seconds; the resumed deletion audit took 32677.27 seconds. Timings are observations, not mathematical evidence.

The main trust paths are intentionally visible:

$$\text{topology JSON} \longrightarrow \text{independent equation manifest} \longrightarrow \text{layered QF_LIA replay},$$

$$\text{topology JSON} \longrightarrow \text{Lean equation builder} \longrightarrow \text{canonical manifest comparison},$$

$$\text{Lean definitions} \longrightarrow \text{generic theorem and one SAT witness} \longrightarrow \text{Lean kernel}.$$

The first path still trusts Z3, Python, the operating system, and the arithmetic encoding. The second catches translation drift but cannot turn a solver status into a theorem. The third has a smaller logical endpoint but does not include the concrete UNSAT instance.

7 Related Work

Seal's construction and the OEIS entry provide historical reverse-and-add context [3, 10]. Our radial frontier is an instance of the familiar principle that bounded separators make structured CSPs tractable [7, 6]; we make no claim that the frontier method itself is new.

For solver-result certification, DRAT-trim checks expressive clausal proofs [11]; LRAT adds hints suitable for simple certified checkers [4]; and FRAT supports communication from a solver to a later elaborator [1]. A future Boolean encoding could use a proof-producing solver such as CaDiCaL [2], followed by one of these checking paths. We did not promote such a translation or trace because it is not required by the direct radial argument and was not validated to the standard of this artifact.

The literature search is documented in `paper/literature-matrix.md`. It did not establish priority for the finite result, so this paper makes no literature-wide novelty claim.

8 Limitations

The following boundaries are part of the result.

- The concrete 409-equation and 408-equation unsatisfiability claims are not kernel-checked and have no independently checked LRAT, DRAT, or FRAT trace.

- Deletion minimality is computationally certified. Only the one neighbor named above is separately checked in Lean.
- This is not a minimum-cardinality claim. Smaller unsatisfiable subsets may exist.
- There is no all-base result and no proof that any particular starting integer is Lychrel. The theorem concerns one fixed base-19 topology and a subset of its column equations.
- The one-deletion audit identifies a single unsatisfiable deletion of $\mathcal{C}$, but this does not justify an unqualified uniqueness claim outside those immediate subsets.
- Runtimes and memory use are platform-dependent and are not part of the mathematical statement.

The most valuable next strengthening would be a compact concrete proof object checked by a substantially smaller independent verifier, or a feasible Lean proof of the concrete frontier closure. Until then, Proposition 1 should be cited with its computational assurance boundary.

References

[1] Seulkee Baek, Mario Carneiro, and Marijn J. H. Heule. A flexible proof format for SAT solver-elaborator communication. *Logical Methods in Computer Science*, 18(2):3:1–3:26, 2022.

[2] Armin Biere and contributors. CaDiCaL sat solver, release 3.0.1, 2026. Public API documentation and release c607304, accessed 8 August 2026.

[3] Kevin S. Brown. Digit reversal sums leading to palindromes, 1996. Transcribes David Seal's January 1996 report; accessed 8 August 2026.

[4] Luís Cruz-Filipe, Marijn Heule, Warren Hunt, Matt Kaufmann, and Peter Schneider-Kamp. Efficient certified RAT verification. In *Automated Deduction – CADE 26*, volume 10395 of *Lecture Notes in Computer Science*, pages 220–236. Springer, 2017.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[6] Rina Dechter and Judea Pearl. Tree clustering for constraint networks. *Artificial Intelligence*, 38(3):353–366, 1989.

[7] Eugene C. Freuder. Complexity of K-tree structured constraint satisfaction problems. In *Proceedings of the Eighth National Conference on Artificial Intelligence*, pages 4–9. AAAI Press, 1990.

[8] London Mathematical Society. Computer aided proofs, November 2024. Policy page, accessed 8 August 2026.

[9] Microsoft Research. Z3 guide: Arithmetic, 2026. Accessed 8 August 2026.

[10] OEIS Foundation Inc. The on-line encyclopedia of integer sequences, a060382, 2026. Accessed 8 August 2026.

[11] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429. Springer, 2014.